ML Concepts — a primer for the mathematician reading this literature

Companion to the survey files in this directory. Audience: someone fluent in linear algebra, geometry, probability, and the basics of optimization, and comfortable with neural-network *inference* (weights, activations, matrix multiplies) — but **not** an ML specialist. The goal is to define, once and precisely, the ML vocabulary that the papers throw around without explanation. Where a term has a clean mathematical definition, it is given. Cross-references point to the companion *Landscape* survey by part number.

This is a living file: as later surveys hit new concepts, append them here rather than re-explaining inline.

1. The object of study: a decoder-only transformer

Everything in this literature concerns **autoregressive decoder-only transformers** (GPT-style). Mathematically, such a model is a function

$$\text{model}: \ (\text{sequence of tokens}) \longmapsto (\text{probability distribution over the next token}).$$

The spaces in play (named once, used throughout). Write $\mathcal{V}$ for the **vocabulary** (a finite set, $|\mathcal{V}| = n_{\text{vocab}}$). The model threads a handful of finite-dimensional real vector spaces:

- **token space** $T = \mathbb{R}^{\mathcal{V}}$ — the free vector space on the vocabulary; a token is a basis vector (§1.1). Logits live in the dual T^*.
- **feature space** $V = \mathbb{R}^{d_{\text{model}}}$ — where each position's *residual stream* vector lives (§1.3). The central space of the whole subject.
- **position space** $P = \mathbb{R}^n$ — one axis per token position in the context; only attention acts on this factor (§1.4).
- **head space** $H = \mathbb{R}^{d_{\text{head}}}$ and **neuron space** $N = \mathbb{R}^{d_{\text{mlp}}}$ — the (low- and high-dimensional) interiors of an attention head (§1.4) and an MLP (§1.5).

A theme worth stating at the outset: the model is built almost entirely from **linear maps between these spaces**. The handful of operations that instead refer to a *chosen basis* of a space — the softmax on T^* (§1.2), the pointwise nonlinearity on N (§1.5), the coordinate-wise normalization on V (§1.6) — are *precisely* the ones that break basis-independence and create the "privileged bases" of §3. So we work with spaces and maps throughout, and descend to coordinates only for those few genuinely basis-dependent operations (where the coordinates are the point).

Adjoints vs. duals (a notational convention). For a linear map $W : A \to B$ we keep two operations distinct, and never write a bare "$^\top$" for either:

- the **dual map** (a.k.a. transpose map) $W^\vee : B^* \to A^*$, $W^\vee(\phi) = \phi \circ W$ — canonical, defined with *no* inner product;
- the **adjoint** $W^* : B \to A$ — defined only once A, B carry inner products, by $\langle Wa, b\rangle_B =$

$\langle a, W^* b\rangle_A$ (the real-scalar specialization of the Hermitian adjoint).

The notation $W^\vee$ for the dual is *not* universal — texts also write ${}^tW$, W', or even W^*; we use $W^\vee$ precisely so that W^* can be reserved for the adjoint. The two coincide in coordinates with the standard inner product — which is exactly why a transpose in the source papers is ambiguous. We resolve each occurrence: W^* **always signals that a metric has been invoked;** $W^\vee$ **never does.**

Consequently a bare "$^\top$" survives in this document in **only one role**: the *literal matrix transpose* inside an expression explicitly marked *"coordinates"* or *"matrix picture"* — an optional, concrete realization of an object we have already named invariantly (e.g. the coordinate form $W_Q^\top W_K$ of the bilinear form W_{QK}, or $XA^\top$ for right-multiplying a position index). It never stands for a map between spaces. A quadratic form $x^\top My$ likewise becomes the canonical pairing $\langle Mx, y\rangle$ with the metric $M : V \to V^*$ written as a map.

1.1 Tokens, vocabulary, embedding, unembedding

- **Token**: the atomic input symbol. Text is chopped by a *tokenizer* into a sequence of tokens (roughly sub-words). The **vocabulary** $\mathcal{V}$ is the set of possible tokens, $|\mathcal{V}| = n_{\text{vocab}}$ (e.g. $\sim$ 50,000); a token is a basis vector of **token space** $T = \mathbb{R}^{\mathcal{V}}$ (a *one-hot* — see box).
- **Embedding**: a linear map $W_E : T \to V$ sending each token to its dense **embedding vector** in the feature space $V = \mathbb{R}^{d_{\text{model}}}$. (As a matrix it is $d_{\text{model}} \times n_{\text{vocab}}$.) Here d_{model} — a few hundred to many thousands — is *the* ambient dimension nearly every paper calls the residual-stream dimension.
- **Unembedding**: a linear map $W_U : V \to T^*$ sending the final stream vector to a **logit covector**, which pairs with each token to score it.
- Often W_E, W_U are independent ("untied"); sometimes **tied** — W_U is the coordinate transpose of W_E, making the logit of token y the inner product $\langle \text{emb}(y), x\rangle_V$ of the stream with y's embedding vector. That pairs two vectors *of* V, so it needs an inner product on V: invariantly $W_U = W_E^\vee$ precomposed with the metric $V \cong V^*$. Even weight-tying thus silently picks a metric (cf. §1.3).

On "one-hot". "One-hot" is ML jargon for a *standard basis vector*: under a fixed enumeration of the vocabulary, token i is the vector $e_i \in \mathbb{R}^{n_{\text{vocab}}}$, all zeros but a single 1 in coordinate i ("exactly one coordinate is hot"; the name comes from digital logic). It is the canonical way to feed a *categorical* variable — a token drawn from an unordered finite set — into linear algebra: encoding "token #4217" as the *scalar* 4217 would impose a spurious order and metric, whereas the e_i are linearly independent and mutually equidistant. It also makes embedding a literal table lookup, since $W_E e_i$ is simply the image of the basis vector e_i (its "i-th column", in matrix terms); dually, the logit for token i is the pairing $\langle \ell, e_i\rangle$ of the logit covector $\ell \in T^*$ with e_i. One nuance worth keeping: although a single token is *one* e_i, the circuit papers freely take *linear combinations* of one-hots (e.g. to reason about "a blend of two tokens"), so the real object is the token space $T = \mathbb{R}^{\mathcal{V}}$ with the tokens as a distinguished basis.

1.2 Logits and softmax

- **Logit**: the model's (unnormalized) score for a candidate next token. The **logit covector** $\ell = W_U\, x_{\text{final}} \in T^*$ assigns token y the value $\ell_y = \langle \ell, e_y \rangle$ (its pairing with the one-hot e_y).
- **Softmax**: the map $T^* \to \Delta(\mathcal{V})$ onto probability distributions over the vocabulary,

$$\text{softmax}(\ell)_y = \frac{e^{\ell_y}}{\sum_{y'} e^{\ell_{y'}}}.$$

This is the first operation that refers to a *basis* — the token basis of T^* — but here the basis is genuinely meaningful (tokens are distinguished), unlike the arbitrary basis of V (§1.3). Two invariances matter later: softmax is unchanged by **adding a constant** to every logit (so a logit is defined only modulo the all-ones covector $\mathbf{1}$ — an affine, not linear, object), and **temperature** scaling $\ell \mapsto \ell/\tau$ sharpens ($\tau \to 0$) or flattens ($\tau \to \infty$) the distribution.

1.3 The residual stream — the central vector space

The defining architectural choice, and the reason transformers are unusually amenable to linear-algebraic analysis. This subsection is worth reading slowly: most of the "circuits" program is downstream of the few exact statements below.

Setup. Fix a token position i. As the input flows through the network, that position carries a vector $x_i^{(k)} \in V$ (the feature space $\mathbb{R}^{d_{\text{model}}}$ of the overview) — the **residual stream** at position i after block k. The model is a sequence of K **blocks** $B_1, \ldots, B_K$ (each block is one attention layer or one MLP; in the finest-grained accounts each individual attention *head* is its own block, since heads add independently — §1.4). The stream is initialized to the embedding plus positional information (§1.7),

$$x_i^{(0)} \;=\; W_E\,(\text{token at position } i) \;+\; (\text{positional term}),$$

and each block updates it through a **residual (skip) connection** — it computes something and *adds the result back* rather than replacing the stream:

$$x_i^{(k)} \;=\; x_i^{(k-1)} \;+\; B_k\big(x^{(k-1)}\big)_i.$$

(Embedding and MLP blocks act position-wise — $B_k(x^{(k-1)})_i$ depends only on $x_i^{(k-1)}$; attention is the *only* block that reads other positions, which is why its argument is the whole $x^{(k-1)}$, not just position i.)

Additivity, stated exactly. Unrolling the recurrence telescopes into a sum:

$$x_i^{(K)} \;=\; x_i^{(0)} \;+\; \sum_{k=1}^{K} B_k\big(x^{(k-1)}\big)_i.$$

The final residual vector is *literally the sum* of the input embedding and every block's output: nothing is ever overwritten, only accumulated. The stream is a **running sum** and each block contributes one summand. This is the precise content of "additivity", and it is **exact and unconditional** — it holds verbatim for every input, whatever the blocks compute.

What is, and is not, linear. A frequent misreading: "linear communication channel" does *not* say the blocks are linear. Each B_k is a genuinely *nonlinear* function of the stream (softmax inside

attention, ReLU/GeLU inside MLPs), so the step map $x^{(k-1)} \mapsto x^{(k)}$ is nonlinear. What is linear is (i) how the blocks' contributions **combine** — by addition — and (ii) how a contribution, once written, is **transported** to later blocks — by the identity map, i.e. carried forward unchanged. The nonlinearity is confined *inside* each block; the bus wiring the blocks together is linear. That separation is exactly what makes the circuits program possible.

Logits are an additive sum of per-block contributions. Since the unembedding W_U is linear, applying it to the telescoped sum distributes over the terms (writing $\ell = W_U x_i^{(K)}$ for the logit vector of §1.2):

$$\ell \;=\; \underbrace{W_U x_i^{(0)}}_{\text{embedding's direct effect}} \;+\; \sum_{k=1}^{K} \underbrace{W_U\, B_k(x^{(k-1)})_i}_{\text{block } k\text{'s direct effect on logits}} \quad .$$

Each summand is a vector in $\mathbb{R}^{n_{\text{vocab}}}$ that can be inspected on its own. Two standard techniques are immediate corollaries:

- **Direct logit attribution** — to ask "which blocks pushed up the logit of output token y?", read off the y-component $\langle e_y,\, W_U B_k(\cdot)_i \rangle$ for each k.
- **The logit lens** — apply W_U to an *intermediate* $x_i^{(k)}$ to see the model's running guess at the next token. This is meaningful precisely because $x_i^{(k)}$ is a partial sum of the very same objects whose full sum produces the final logits.

Reading and writing subspaces. A block touches the stream through two linear maps: it *reads* via an input map W_{in} (e.g. W_Q, W_K, W_V for attention, or the MLP's first map) and *writes* via an output map W_{out} (e.g. W_O, or the MLP's second map). What follows is what the stream's geometry says, exactly, about how blocks can and cannot interact — and it is a good place to see which statements survive the gauge freedom and which are metric artifacts.

What a block produces and sees — metric-free. Locate the two maps by their relation to the stream space V: the write map W_{out} has V as its **destination**, the read map W_{in} has V as its **source**. Two statements then need no inner product:

- **Write.** The vector a block adds lies in $\mathrm{im}(W_{\text{out}}) \subseteq V$, its **write subspace**, of dimension $\leq \mathrm{rank}(W_{\text{out}}) \leq d_{\text{head}}$ (resp. $\leq d_{\text{mlp}}$).
- **Read.** The block depends on the stream vector x only through $W_{\text{in}}x$, i.e. only through the class $\bar{x} \in V/\ker(W_{\text{in}})$. Inputs agreeing modulo $\ker(W_{\text{in}})$ are indistinguishable to it; equivalently, the block is **invariant to adding anything in** $\ker(W_{\text{in}})$ to the stream.

Note the asymmetry in *kind*, and that read and write live in **dual spaces**. The map-native **write** object is an honest **subspace of** V ($\mathrm{im}\, W_{\text{out}}$). The map-native **read** object is a **quotient of** V ($V \twoheadrightarrow V/\ker W_{\text{in}}$) — equivalently, dually, the **subspace of covectors** $\mathrm{im}(W_{\text{in}}^{\vee}) \subseteq V^*$ that the block actually evaluates on the stream. So writes are vectors in V and reads are covectors in V^*; a metric is exactly what one would need — illegitimately — to put them in the same space. Only the quotient / covector description is forced on us if we refuse a metric.

Turning the read side into a subspace of V needs a metric — and that is the catch. To name the read side as a subspace of V (rather than a quotient, or a subspace of V^*) one must choose a complement to $\ker(W_{\text{in}})$. The standard inner product supplies one, $V = \ker(W_{\text{in}}) \oplus \ker(W_{\text{in}})^{\perp}$, and then the block depends on x only through the orthogonal projection $\Pi_{\text{read}}\, x$ onto $\ker(W_{\text{in}})^{\perp}$, since $W_{\text{in}}x = W_{\text{in}}(\Pi_{\text{read}}x)$. But §1.3 has **no privileged inner product**, and this complement is

a *metric artifact*. Under the gauge $x \mapsto Mx$ (so $W_{\text{in}} \mapsto W_{\text{in}}M^{-1}$) compare the three objects:

- the **kernel** transforms covariantly, $\ker \mapsto M \ker$ (a subspace of V);
- its metric-free dual, the **annihilator** $(\ker W_{\text{in}})^0 = \text{im}(W_{\text{in}}^\vee) \subseteq V^*$, transforms by the inverse **dual map** $(M^\vee)^{-1}$;
- the **orthogonal complement** $\ker^\perp$ transforms by the inverse **adjoint**, $(M \ker)^\perp = (M^*)^{-1}(\ker)^\perp$ — *not* $M(\ker)^\perp$.

The complement and the annihilator agree only when M is orthogonal ($M^* = M^\vee$ under the identification). So the kernel, the quotient $V/\ker$, and the annihilator are gauge-natural; the read subspace $\ker^\perp \subseteq V$ — the "row space", obtained by dragging the annihilator across the metric isomorphism $V^* \cong V$ — is not.

Non-interaction, formalized — and gauge-invariant. When does a writer A leave a reader B **completely** unaffected, on every input? A contributes $W_{\text{out}}^A(\,\cdot\,)$ to the stream that B reads through W_{in}^B, so A's entire effect on B's read is the composite $W_{\text{in}}^B W_{\text{out}}^A(\,\cdot\,)$. Hence

$$\underbrace{A \text{ never affects } B}_{\text{for every input}} \iff \text{im}(W_{\text{out}}^A) \subseteq \ker(W_{\text{in}}^B) \iff \underbrace{W_{\text{in}}^B\, W_{\text{out}}^A = 0}_{\text{their virtual weight vanishes}} .$$

All three are inner-product-free, and the virtual weight is **gauge-invariant** ($W_{\text{in}}^B M^{-1} \cdot M W_{\text{out}}^A = W_{\text{in}}^B W_{\text{out}}^A$). This is the *correct* formalization of "A and B occupy orthogonal channels": not metric orthogonality of subspaces, but **image contained in kernel**. Degree of interaction is then the size of $W_{\text{in}}^B W_{\text{out}}^A$ — its rank, or its norm once a metric is fixed.

Capacity, and why metric orthogonality returns as "interference". The stream offers only $\dim V = d_{\text{model}}$ independent directions. Several writers' outputs can be linearly disentangled by a downstream reader exactly when their write subspaces are in **direct sum**, $\text{im}\, W_{\text{out}}^{A_1} \oplus \cdots \oplus \text{im}\, W_{\text{out}}^{A_k} \subseteq V$ (then projections Π_j isolate each), which forces $\sum_j \dim \text{im}\, W_{\text{out}}^{A_j} \leq d_{\text{model}}$. But the model traffics in far more *features* than d_{model} (§3), so exact direct sums are impossible: write subspaces must overlap, and the virtual weights $W_{\text{in}}^B W_{\text{out}}^A$ between blocks that "should" be unrelated are forced *nonzero*. That residual cross-talk **is** *interference* (§3). Choosing the data-induced inner product (the one training commits to, §2), it is small precisely when the relevant write/read directions are *nearly orthogonal* — and the packing problem becomes "the feature Gram matrix is close to I" (§4, the *Landscape* survey, Part III). So metric orthogonality is not fundamental, but it is the right *approximate* language once a model has settled on a working inner product. (For attention, the overlap of different heads' write subspaces is itself a studied phenomenon, "attention superposition" — Jermyn et al. [11].)

A concrete channel. The induction circuit above is one read/write channel made explicit. A **previous-token head** writes, into its write subspace $\text{im}(W_O^{\text{prev}})$, a vector encoding "the token just before me was t". Layers later an **induction head**'s key map W_K^{ind} is tuned to read that very subspace — i.e. the virtual weight $W_K^{\text{ind}} W_O^{\text{prev}} \neq 0$ — so it can match the current token against each earlier token's predecessor. The *same* induction head's query map reads a different part of the stream (current-token identity), and the many other heads writing elsewhere are precisely those whose images W_K^{ind} (approximately) annihilates: they share the bus without colliding with this channel.

So the residual stream is a shared memory bus of width d_{model}: blocks deposit into write subspaces and later blocks read out of the quotients dual to their read maps, with the virtual weight $W_{\text{in}}^B W_{\text{out}}^A$ the exact, gauge-invariant measure of how loudly A speaks on B's channel.

Virtual weights. Suppose an earlier block A writes via W_{out}^A and a later block B reads via W_{in}^B. Because the identity carries A's output forward untouched, the portion of B's read that is *due to* A is governed by the single composite linear map

$$W_{\text{virtual}} := W_{\text{in}}^B W_{\text{out}}^A,$$

the **virtual weight** from A to B. It is computable directly from the weights, with no reference to any input, and it exists *even though A and B are not adjacent and the architecture contains no explicit weight connecting them* — the residual connections supply the implicit identity wiring. Concretely: a *previous-token head* writes a "the token before me was t" signal into some write subspace; several layers later an *induction head* reads that subspace through its W_K; the virtual weight $W_K^{\text{ind}} W_O^{\text{prev}}$ is exactly the channel by which the first head speaks to the second, and analysing it is how Elhage et al. [1] reverse-engineers the induction circuit (§3, the *Landscape* survey, Part I).

Why this forces superposition. The number of distinct signals the blocks collectively want to read and write vastly exceeds d_{model} — a single MLP already exposes $\sim 4\, d_{\text{model}}$ neurons, each a potential feature. The stream is therefore a **bandwidth bottleneck**: many features must share the same d_{model} dimensions, which is precisely the pressure toward *superposition* (§3, and the *Landscape* survey, Part I). Stream vectors forced to carry far more than d_{model} signals are sometimes called "bottleneck activations" and are expected to be especially hard to interpret.

No privileged basis. Finally, because every read and write is an arbitrary linear map, one may apply *any* automorphism $M \in GL(V)$ to the entire stream and absorb it into the weights — replace each read map W_{in} by $W_{\text{in}} M^{-1}$ and each write map W_{out} by $M\, W_{\text{out}}$ — for a bit-identical function. The stream thus has **no privileged basis**: its individual coordinates carry no intrinsic meaning, and the space comes with a $GL(V)$ **gauge symmetry**. (A pointwise nonlinearity, as on MLP neurons, *would* break this and single out a basis — §1.5.) The consequences of this gauge freedom — in particular that within-stream quantities like cosine similarity are not canonically defined — are the subject of the *Landscape* survey, Part III.

1.4 Attention

The block that moves information *between* token positions, and the **only** one acting on the position factor P (embedding and MLP act on the feature space V alone, position by position). Assembling all positions, the residual stream at a layer is an element $X \in V \otimes P$ — concretely the $d_{\text{model}} \times n$ matrix whose j-th column is the stream vector x_j at position j.

One head is four linear maps through the low-dimensional head space $H = \mathbb{R}^{d_{\text{head}}}$ (with $d_{\text{head}} \ll d_{\text{model}}$):

$$W_Q, W_K, W_V : V \to H, \qquad W_O : H \to V.$$

The head computes, in three steps:

1. **Scores (a bilinear form on V, landing in the dual).** Query and key maps send the stream to H-valued fields $W_Q X, W_K X$, and the score of destination i attending to source j is the **head-space inner product** $\langle W_Q x_i,\, W_K x_j \rangle_H$ (an *adjoint* pairing on H). Invariantly this is a single learned **bilinear form** on V,

   $$W_{QK} : V \to V^*, \qquad W_{QK} = W_Q^\vee\, g_H\, W_K \quad (\text{coordinates: } W_Q^\top W_K),$$

   where $g_H : H \xrightarrow{\sim} H^*$ is the inner product on H and $W_Q^\vee$ the dual map. The score is then the **canonical pairing** $\langle W_{QK}\, x_j,\ x_i \rangle$ of a covector with a vector — needing **no inner product**

on V. The lone transpose (in the coordinate form) marks precisely that W_{QK} lands in the dual V^*; and the one metric used, g_H on the *head* space, is **internal** — a change of basis on H rescales W_Q, W_K oppositely and leaves W_{QK} fixed. The form need not be symmetric (query- and key-roles differ), so it distinguishes "i attends to j" from "j attends to i". Evaluated on every pair of positions it gives the score array S — itself a bilinear form on the position space P.

1. **Pattern (the only nonlinearity).** Normalize each row of S (scaled by $1/\sqrt{d_{\text{head}}}$, and with a *causal mask* forbidding attention to future positions, $j > i$) through a softmax:

$$A \;=\; \text{rowsoftmax}(S/\sqrt{d_{\text{head}}} \;+\; \text{mask}) \;\in\; \text{End}(P).$$

   A is **row-stochastic** (rows are probability vectors) and, under the causal mask, lower triangular. Geometrically it is an **averaging / Markov operator on** P: each destination position will receive a *convex combination* of source contributions. Through S, the operator A depends on X — and this dependence (softmax of a quadratic form in X) is the sole source of nonlinearity in the entire head.

1. **Output (move values, transform them).** With $A \in \text{End}(P)$ the attention pattern, each destination collects the A-weighted average of values and maps it back to V. Coordinate-free, the head **factors across the two tensor factors** of $V \otimes P$:

$$\text{head} \;=\; W_{OV} \otimes A, \qquad W_{OV} := W_O W_V : V \to V,$$

   with W_{OV} on the **feature** factor (*what* is moved and how it is transformed) and A on the **position** factor (*where* it moves) — independent actions, given A. This is the central decomposition of Elhage et al. [1]. (In the $d_{\text{model}} \times n$ matrix picture, $\text{head}(X) = W_{OV}\, X\, A^\top$; that transpose is **neither a dual nor an adjoint** — only the bookkeeping of right-multiplying the position index.)

The two recurring circuits, both of rank $\leq d_{\text{head}}$ (they factor through H), and — the point worth absorbing — of different *type*:

- **QK circuit** $W_{QK} : V \to V^*$, a **bilinear form** ($(0,2)$-tensor in $V^* \otimes V^*$) governing *where* a head attends. It lands in the dual — hence the transpose in the coordinate form $W_Q^\top W_K$ — and feeds the nonlinear softmax.
- **OV circuit** $W_{OV} = W_O W_V : V \to V$, an **endomorphism** (in $V^* \otimes V$) governing *what* is written when a position is attended to. No transpose appears — no dual is involved. (Its image is the head's **write subspace** of §1.3; the eigenstructure of the endomorphism W_{OV}, read in the token basis, diagnoses e.g. a "copying" head — positive real eigenvalues mean attending to token t raises t's own logit.)

That one is a **bilinear form** and the other an **endomorphism** is the where/what distinction made structural: *where* pairs two stream vectors into a number (a score), so it is a $V \to V^*$ object; *what* sends a stream vector to a stream vector to add, so it is a $V \to V$ object. The presence of a transpose in W_{QK} and its absence in W_{OV} is exactly that type signature.

Freezing the attention pattern — the move that makes circuits tractable

The head map (matrix picture) $X \mapsto W_{OV}\, X\, A(X)^\top$ is nonlinear, but in a very controlled way: **the only nonlinear dependence on X is through $A = A(X)$.** Hold A fixed at a constant matrix A_0 and the residual map

$$X \longmapsto (W_{OV} \otimes A_0)\, X \qquad \text{(matrix picture: } W_{OV}\, X\, A_0^\top\text{)}$$

is **exactly linear** — it is a single fixed linear operator on $V \otimes P$, the tensor product of two constant linear maps.

The conceptual content: the stream X plays *two roles* in a head. It determines the **routing** (through the QK path $X \mapsto A$) and it supplies the **content** that gets moved (through the value/OV path). "Freezing A" means computing the routing $A_0 = A(X_0)$ on some reference input X_0, then *detaching* it — regarding the head as a function of the content-carrying copy of X alone, with routing held fixed. Under that split the head is the constant linear operator above.

Two properties make this powerful, and distinguish it sharply from a Taylor/Jacobian linearization:

- **Exact at the operating point, not first-order.** At $X = X_0$ we have $A_0 = A(X_0)$, so $W_{OV} X_0 A_0^\top$ equals the head's *true* output on X_0 — exactly, with no remainder. Freezing is not an approximation of the value; it is an exact rewriting that happens to be linear in the content direction. (Contrast §3's *local linearization*, which matches the true map only to first order and drifts away from X_0.)
- **Globally linear in content.** As a function of the value-path input (routing fixed), it is linear *everywhere*, not just near X_0.

What you give up is precisely the dependence of routing on the input — the derivative of A through the QK circuit. That discarded piece is the genuinely nonlinear, and empirically the *harder*, half of attention: explaining **why** a head attends where it does. The field handles this by **splitting the head in two** along exactly this seam — study the OV circuit with A frozen (clean linear algebra), and study the QK circuit's routing separately (still largely open; see Kamath et al. [12], Jermyn et al. [11]).

Why this "makes circuit analysis tractable": with every head's A frozen to constants, an entire attention layer becomes a fixed linear operator on $V \otimes P$. Composing such layers with the (linear) residual stream of §1.3, the embedding W_E, and the unembedding W_U yields an **end-to-end linear map from input tokens to logits**, which can be multiplied out into a sum of interpretable paths (*path expansion*; Elhage et al. [1], the *Landscape* survey, Part I). The OV circuit then says, for the established routing, exactly which token-content is copied or transformed into which logit direction — a question of reading eigen/singular structure of fixed matrices. The one caveat: A is input-dependent, so the frozen-linear model is valid *for the input it was frozen on*; on a different input the pattern, hence the linear operator, changes. (This per-input frozen-attention model is exactly the "local replacement model" of the circuit-tracing program — §3, the *Landscape* survey, Group 4.)

The token-to-token circuits, and a 1-layer model attending by hand

The path expansion of a **1-layer attention-only** model (*Landscape* survey, Group 1) conjugates the two circuits above by the embedding and unembedding to get two honest $n_{\text{vocab}} \times n_{\text{vocab}}$ **token-to-token** matrices: the **QK circuit** $W_E^\vee W_{QK} W_E$ (coordinates $W_E^\top W_{QK} W_E$), scoring how much

a query *token* wants to attend to a key *token*, and the **OV circuit** $W_U W_{OV} W_E$, recording how an attended token shifts the output logits. Each has rank $\leq d_{\text{head}}$ (it factors through H). The worked example below instantiates both on numbers small enough to run softmax by hand; it is the concrete companion to the where/what story above.

Worked example: a skip-trigram head.
What the model does, and what this head is for. The whole model is a **next-token predictor**: feed it a sequence of tokens (a *prefix* of text) and it returns a probability distribution over the single token that comes next. You *read* a piece of text by running the model once over it; you *generate* text by sampling the predicted token, appending it to the prefix, and running again — one token at a time, left to right. Our toy head is built to handle one scrap of English, the phrase "`keep ... in mind`". Concretely we will hand it the prefix "`keep this in`" and ask it to predict the next token, which should be "`mind`" (the earlier tokens we simply take as given).
Three tokens, three different roles — and "attend" is not "predict". The step that trips up newcomers is that **three distinct tokens** are in play, doing three different jobs:

- the **source**, a.k.a. the **key** the head lands on: `keep`, sitting at an *earlier* position — what the head *looks back at*;
- the **destination**, a.k.a. the **query**: `in`, the *current* position, the one whose next token we are predicting — what does the *looking*;
- the **output**: `mind`, the token whose logit the head ultimately *raises* — the actual prediction.

So "query `in` wants key `keep`" means: *when the current token is* ***`in`****, the head reaches backward through the context and pulls information out of the earlier position holding* ***`keep`****.* It does **not** mean the model outputs `keep`. Attention only *moves information between positions*; a *separate* map (the OV circuit) then turns the retrieved `keep` into the prediction `mind`. Nothing is reversed in text order: `keep` occurs first, `in` later, and `mind` is what comes after `in` — "`keep this in mind`". (Your worry — "does this generate `keep` after `in`?" — conflates the *key* the head reads with the *output* it writes; they are different tokens.)
The setup. Take $n_{\text{vocab}} = 4$ with token basis ordered $(\texttt{keep}, \texttt{in}, \texttt{mind}, \texttt{this})$. To strip away everything but the two circuits, identify $V \cong T$ via $W_E = I$ (each token embeds to its own basis vector) and tie the unembedding $W_U = I$ (coordinates). Then $W_U W_E = I$ — the *direct path* $\text{Id} \otimes W_U W_E$ just copies the current token's one-hot to the logits (the bigram baseline) — and the two token-to-token circuits are *literally* W_{QK} and W_{OV} read in the token basis. Give the single head these two rank-1 circuits (rows/cols ordered as above):

$$\underbrace{W_E^\top W_{QK} W_E}_{\text{QK circuit}} = 4\,(\mathbf{e}_{\texttt{in}})(\mathbf{e}_{\texttt{keep}})^\top = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 4 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix},$$

$$\underbrace{W_U W_{OV} W_E}_{\text{OV circuit}} = 5\,(\mathbf{e}_{\texttt{mind}})(\mathbf{e}_{\texttt{keep}})^\top = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 5 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

Read in those three roles: the QK row for *query* `in` has its one nonzero entry in the *key* column `keep` (value 4) — the "*when current token is* ***`in`****, look back at* ***`keep`***" instruction; the OV column for *source* `keep` has its one nonzero entry in the *output* row `mind` (value 5) — the "*having found* ***`keep`****, push up the prediction* ***`mind`***" instruction. Both are rank 1, so $d_{\text{head}} = 1$ realises them,

e.g. W_K : `keep` $\mapsto 1$ (else 0), W_Q : `in` $\mapsto 4$, and W_V : `keep` $\mapsto 1$, $W_O : 1 \mapsto 5\,\mathbf{e}_{\texttt{mind}}$.
Run it on the prefix `keep, this, in` (positions $1, 2, 3$), asking for the token after position 3. The destination is position 3, query token `in`; the causal mask allows keys at positions $1, 2, 3$, with key tokens `keep, this, in`.
Step 1 — scores. Read row `in` of the QK circuit against each key token:

$$s_1 = \langle \texttt{in} \mid \texttt{keep} \rangle = 4, \qquad s_2 = \langle \texttt{in} \mid \texttt{this} \rangle = 0, \qquad s_3 = \langle \texttt{in} \mid \texttt{in} \rangle = 0.$$

Step 2 — pattern (the one nonlinearity). Row-softmax over positions $1, 2, 3$ (folding $1/\sqrt{d_{\text{head}}}$ into the 4):

$$A_{3,\cdot} = \text{softmax}(4, 0, 0) = \frac{(e^4,\ 1,\ 1)}{e^4 + 2} \approx (0.965,\ 0.018,\ 0.018).$$

The destination puts $\approx$96.5% of its attention on position 1 — it has **attended to** `keep`.
Step 3 — output (OV moves the content). The head adds $\sum_j A_{3,j}\, W_{OV}\, \mathbf{e}_{(\text{token } j)}$. In the token basis $W_{OV}\mathbf{e}_{\texttt{keep}} = 5\,\mathbf{e}_{\texttt{mind}}$ and $W_{OV}\mathbf{e}_{\texttt{this}} = W_{OV}\mathbf{e}_{\texttt{in}} = 0$, so the head's contribution to the position-3 logits is

$$0.965 \cdot 5\,\mathbf{e}_{\texttt{mind}} \ +\ 0.018 \cdot 0 \ +\ 0.018 \cdot 0 \ \approx\ 4.82\,\mathbf{e}_{\texttt{mind}}.$$

Adding the direct path (+1 to `in`), the logit vector is $\approx$ (`keep` : 0, `in` : 1, `mind` : 4.82, `this` : 0), so the model predicts `mind` — completing "`keep this in mind`". We have read the **skip-trigram** [`keep`] ... [`in`] $\to$ [`mind`] straight off the two matrices: a nonzero QK entry (*where*: `in` attends back to `keep`) composed with a nonzero OV entry (*what*: an attended `keep` boosts `mind`).
Why the factoring is lossy (the "skip-trigram bug"). Suppose the same head must also do [`keep`] ... [`at`] $\to$ [`bay`]. That means QK row `at` also points at `keep`, and OV column `keep` also boosts `bay`. But the head's effect on a (destination, out) pair factors as (attention to source) $\times$ (OV[out, source]) — the destination enters only through QK, the output only through OV, coupled solely by the *shared* source token. With `keep` as source attracting both destinations *and* boosting both outputs, all four combinations fire, including the unwanted [`keep`] ... [`in`] $\to$ [`bay`]. No choice of weights separates them: that is the documented bug, and it is visible here as "column `keep` of the OV circuit is one vector, shared by every destination that attends to `keep`."

Multi-head. Several heads run in parallel per layer over the *same* stream; their outputs are **summed** back in, so the layer is $\sum_h W_{OV}^h \otimes A^h$ (matrix picture $\sum_h W_{OV}^h\, X\, (A^h)^\top$). (Implementation-wise the per-head H_h are concatenated and W_O is correspondingly block-structured, but additivity across heads — like additivity across blocks in §1.3 — is the mathematically load-bearing fact.) Because each W_{OV}^h, W_{QK}^h has rank $\le d_{\text{head}}$, a head reads from and writes to only a d_{head}-dimensional slice of the d_{model}-dimensional stream.

1.5 MLP (feed-forward) layer

The other block type: a two-layer perceptron acting on the feature space V, **independently at each position**, through a high-dimensional **neuron space** $N = \mathbb{R}^{d_{\text{mlp}}}$ (typically $\dim N \approx 4 \dim V$):

$$\text{MLP} = W_{\text{out}} \circ \sigma \circ (W_{\text{in}}(\cdot) + b_{\text{in}}) + b_{\text{out}}, \qquad W_{\text{in}} : V \to N, \quad W_{\text{out}} : N \to V,$$

with σ a **pointwise** nonlinearity on N (historically ReLU $\max(0, \cdot)$; modern: **GeLU** $x\,\Phi(x)$, a smooth ReLU; **SoLU** $x \odot \mathrm{softmax}(x)$, see the *Landscape* survey). The coordinates of N are the **neurons**. MLPs hold $\sim 2/3$ of a transformer's parameters and are far less understood than attention.

Because σ acts coordinate-wise, it is the operation that refers to — and thereby **privileges** — the neuron basis of N. Unlike the feature space V (no privileged basis, §1.3), N comes with a distinguished basis, which is why early interpretability hoped to read meaning off individual neurons — and why *superposition* (§3) is the obstruction when meaning fails to align with that basis.

1.6 LayerNorm / RMSNorm

A near-normalization of the stream vector, applied per position before a block reads it; this is the one place the coordinates of V are genuinely referenced.

- **LayerNorm**: $\mathrm{LN}(x) = \gamma \odot \dfrac{x - \mu(x)\,\mathbf{1}}{\varsigma(x)} + \beta$, where $\mu(x)$ and $\varsigma(x)$ are the mean and standard deviation of the coordinates of x, and γ, β are learned. Two steps are basis-dependent: mean-subtraction is the orthogonal projection *off the all-ones line* $\mathbb{R}\mathbf{1}$ (privileging that direction), and the learned $\gamma \odot (\cdot)$ is a fixed diagonal map in the coordinate basis. The sole nonlinearity is the division by the scalar $\varsigma(x) = \|x - \mu(x)\mathbf{1}\|/\sqrt{\dim V}$, which is often "folded in" or *frozen* (treated as a constant) during analysis, leaving LN affine.
- **RMSNorm**: drops the mean-centering — $x \mapsto \gamma \odot x/\,\mathrm{RMS}(x)$, $\mathrm{RMS}(x) = \|x\|/\sqrt{\dim V}$. Apart from the learned diagonal γ, this depends on x only through $\|x\|$ and the direction $x/\|x\|$, hence is **rotation-invariant** — whereas LayerNorm's mean-subtraction privileges $\mathbf{1}$. Used in experiments that need exact basis-independence (e.g. the rotation tests in Elhage et al. [5]).

1.7 Positional information

Tokens have no intrinsic order, so position is injected: learned **positional embeddings** added to the stream, or **RoPE** (rotary positional embeddings), which rotates q, k by a position-dependent angle — meaning some positional information lives *in the QK bilinear form*, not in the residual stream (a caveat for attribution methods).

2. Training

- **Loss**: training minimizes **cross-entropy** $-\log \mathbb{P}_{\text{model}}(\text{actual next token})$, averaged over a corpus. Reported in **nats** (natural-log units). Equivalent to maximizing log-likelihood; the model is a density estimator over text.
- **Gradient descent / SGD**: parameters updated by $-\eta \nabla_\theta L$ on mini-batches.
- **Adam / AdamW**: the optimizer of choice. It keeps **per-coordinate** running estimates of the gradient's first and second moments and normalizes the update coordinate-wise. The "**W**" adds decoupled **weight decay** (an ℓ_2 shrinkage of weights each step). *Per-coordinate normalization is basis-dependent* — this is the suspected cause of basis privilege in the residual stream (see the *Landscape* survey).

- **Weight decay / regularization**: penalties added to the loss to bias toward small/simple weights — ℓ_2 (ridge), ℓ_1 (sparsity-inducing), or ℓ_0 (literal count of nonzeros, NP-hard, approximated by ℓ_1). Central to dictionary learning (SAEs) and to grokking.
- **Epoch / full-batch / learning-rate schedule**: an epoch is one pass over the data; toy models often use full-batch (exact gradient); learning rates are typically warmed up then decayed.
- **Pretraining vs finetuning**: *pretraining* is the big next-token run on web-scale text; *finetuning* (incl. **SFT**, **RLHF** — reinforcement learning from human feedback) adapts the pretrained model to be a helpful assistant, refuse harmful requests, etc.
- **Scaling laws**: empirically, loss falls as a power law in compute/parameters/data. Used here to budget SAE training too.

2.1 Two training phenomena that are themselves objects of study

- **Grokking**: on small algorithmic tasks, a network first **memorizes** the training set (train loss $\to 0$, test loss high), then — often long after — **suddenly generalizes** (test loss drops). See the *Landscape* survey, Part II, Group 5, and Nanda et al. [7].
- **Double descent**: test error, as a function of model size or dataset size, can go down, then **up** (around the interpolation threshold), then down again — contradicting the classical U-shaped bias–variance picture. See Henighan et al. [6].

3. Interpretability-specific vocabulary

This is the vocabulary invented by the literature itself; the survey's mathematical arc is mostly about these.

- **Feature**: the unit of "meaning." The dominant working definition (the **Linear Representation Hypothesis**) is that a feature is a **direction** (a unit vector, or a ray/cone) in some activation space, and that the activation decomposes as a sparse linear image $x \approx b + D\,f(x)$ of a nonnegative sparse code $f(x) \in \mathbb{R}^F_{\geq 0}$ under a **dictionary** map $D : \mathbb{R}^F \to V$ whose values $d_i := De_i$ on the standard basis are the feature directions (equivalently $x \approx b + \sum_i f_i(x)\, d_i$). A more recent minority view (`when-models-manipulate-manifolds`) is that a feature can be a **curved low-dimensional manifold**, not a single direction. See the *Landscape* survey, Part IV.
- **Activation**: the actual numerical vector at some site (residual stream at layer k, MLP neurons, attention values) for a given input. Contrast **weights** (fixed parameters).
- **Monosemantic vs polysemantic**: a neuron/feature is *monosemantic* if it responds to one human-interpretable concept, *polysemantic* if it fires for several unrelated ones. Polysemanticity is the empirical norm for neurons and is the symptom of superposition.
- **Privileged basis**: a basis of an activation space that is *singled out by the architecture*. Pointwise nonlinearities (ReLU on MLP neurons) create one; purely linear surroundings (the residual stream, attention value space) do not. Formally: a representation space U is *non-privileged* if inserting M before and M^{-1} after leaves the function unchanged for any automorphism $M \in GL(U)$ — i.e. it has a $GL(U)$ **gauge symmetry**. This gauge idea is a unifying thread in the *Landscape* survey.
- **Superposition**: a layer of width d representing $\gg d$ features as an **overcomplete** set of almost-orthogonal directions, made recoverable by **sparsity** (few features active at once). The

enabling mathematics: **Johnson–Lindenstrauss** (see §4) and **compressed sensing**. For a linear code given by an embedding map $W : e_i \mapsto d_i$ (feature-coordinate space $\to$ the width-d space), superposition is present **iff the feature directions $\{d_i\}$ are linearly dependent** — equivalently W is non-injective ($\ker W \neq 0$), a metric-free condition; equivalently (invoking the metric) W^*W is singular — which is forced once there are more features than dimensions. See Elhage et al. [2].

- **Interference**: the off-target inner products $\langle d_i, d_j \rangle$, $i \neq j$, between feature directions — the "noise" superposition pays for packing. The off-diagonal of the Gram matrix D^*D (the adjoint D^* makes the metric-dependence explicit; §4). "**Interference weights**" are the spurious effective weights this induces between features (Anthropic Interpretability Team [9]).
- **Sparse Autoencoder (SAE)**: the workhorse tool. A wide autoencoder reconstructing activations $x \in V$ through an **overcomplete** ($F \gg \dim V$) sparse code in $\mathbb{R}^F$:
  - encoder $f = \sigma(W_{\text{enc}}(x) + b_{\text{enc}})$, with $W_{\text{enc}} : V \to \mathbb{R}^F$ (σ = ReLU or **JumpReLU**),
  - decoder $\hat{x} = b_{\text{dec}} + D\, f(x)$, with the **dictionary** map $D = W_{\text{dec}} : \mathbb{R}^F \to V$,
  - loss $\mathbb{E}\big[\, \|x - \hat{x}\|_2^2 + \lambda \sum_i f_i(x)\, \|De_i\|_2 \big]$ (reconstruction + decoder-norm-weighted ℓ_1 sparsity). The images De_i are the recovered **feature directions**; this is **dictionary learning** with a learned overcomplete dictionary D. See the *Landscape* survey, Part II, Group 3.
- **Dictionary learning / sparse coding**: classical problem (Olshausen–Field, k-SVD): given data, find an overcomplete dictionary D and sparse codes such that $x \approx Df$, f sparse. SAEs are a neural, amortized version.
- **Transcoder**: like an SAE, but instead of reconstructing its own input it predicts a *later* activation (e.g. an MLP's **output** from its **input**) — so it models **computation**, not just representation. **Cross-layer transcoder (CLT)**: features at layer ℓ write to all later layers. **Crosscoder**: one shared dictionary reading/writing multiple sites (layers, or even different models) at once.
- **JumpReLU**: a thresholded ReLU (0 below a learned threshold, identity above) used to get crisper sparsity.
- **Dead feature**: an SAE feature that (almost) never activates — wasted capacity; mitigated by "resampling."
- **Feature splitting**: as dictionary width F grows, one coarse feature resolves into several finer ones (e.g. "San Francisco" $\to$ many SF-related features). Suggests a hierarchy / no canonical feature count.
- **Probe (linear probe)**: a linear (or logistic) classifier given by a **covector** $w \in V^*$, reading off the canonical pairing $\langle w, x \rangle = w(x)$ — no metric needed. Tests whether information is **linearly decodable**. A "concept direction" is properly this covector w; calling it a *direction* (a vector in V) silently transports it across the metric isomorphism $V^* \cong V$.
- **Steering / activation steering / clamping**: *intervening* at inference by adding a vector ($x \mapsto x + \alpha v$) or fixing a feature's value, to causally change behavior. Tests whether a direction is causal, not merely correlational.
- **Circuit**: a subgraph of the model's computation (specific heads/features and the connections among them) implementing a specific behavior. The "circuits" program reverse-engineers these.
- **Ablation**: deleting a component (set its output to **zero** — *zero ablation*, or to its dataset **mean** — *mean ablation*) and measuring the behavioral change. A causal-importance test.
- **Activation patching / path patching / attribution patching**: causal-intervention techniques. *Activation patching* copies an activation from a "clean" run into a "corrupted" run (or vice versa) and measures the effect. *Path patching* restricts this to specific paths between com-

ponents. *Attribution patching* is the cheap **linear (gradient) approximation** of patching: effect $\approx \nabla_x(\text{metric}) \cdot \Delta x$.

- **Attribution graph**: a per-prompt causal graph (nodes = features/tokens/logits, edges = linear attributions) extracted from a locally-linearized replacement model. See Ameisen, Lindsey, et al. [8] and Lindsey et al. [13].
- **Faithfulness / completeness / minimality**: the validation criteria for a claimed circuit C relative to the full model M (formalized in Wang et al. [4]). Roughly: *faithful* = C alone reproduces M's behavior; *complete* = C contains all the relevant components; *minimal* = no component of C is redundant. **Mechanistic faithfulness** (a distinct, deeper notion, Anthropic Interpretability Team [10]) = the approximation implements the *same algorithm*, not merely the same input–output function.
- **In-context learning (ICL)**: the ability to learn a pattern from the prompt itself (no weight updates), e.g. few-shot examples. Largely attributed to **induction heads**.
- **Induction head**: a two-head circuit implementing the copy-completion rule $[A][B] \dots [A] \to [B]$ (and fuzzy versions). The canonical worked example of the whole framework. See Olsson et al. [3].

4. Mathematical facts the papers lean on (you know these, but here's the ML framing)

- **Johnson–Lindenstrauss lemma**: in $\mathbb{R}^d$ one can place **exponentially many** — $e^{\Omega(\varepsilon^2 d)}$ — **unit vectors that are pairwise almost orthogonal** ($|\langle u_i, u_j\rangle| < \varepsilon$). (Equivalently: any finite set of points embeds linearly into dimension $O(\varepsilon^{-2}\log(\#\text{points}))$ with pairwise distances preserved to $1 \pm \varepsilon$.) This is the existence proof for superposition — the feature space V has room for vastly more than $\dim V$ nearly-orthogonal feature directions.
- **Compressed sensing**: a k-sparse vector in $\mathbb{R}^F$ is recoverable from $\Omega(k\log(F/k))$ linear measurements, provided the measurement map is "incoherent" (RIP / restricted isometry; low **coherence** $\max_{i\neq j}|\langle d_i, d_j\rangle|$). Read into the architecture: a model can pack F sparse features (at most k active at once) into a feature space of dimension $\dim V \gtrsim k\log(F/k)$ and still recover them — and an SAE is the recovery algorithm. This *upper-bounds* how much superposition is possible.
- **Coherence / Gram matrix**: for a dictionary map $D : \mathbb{R}^F \to V$ with unit-norm columns $d_i = De_i$, the **Gram matrix** $G = D^*D$ (an endomorphism of the code space $\mathbb{R}^F$, entries $G_{ij} = \langle d_i, d_j\rangle$) is built from the **adjoint** D^*, so it depends on the chosen inner product on V. Its off-diagonal is interference, its diagonal is 1; near-orthogonality means $G \approx I$. Almost every quantitative claim in this literature is, under the hood, a statement about G — and hence about that silent choice of metric (see the *Landscape* survey, Part III).
- **Thomson problem / packing**: minimizing pairwise interaction energy of points on a sphere; its discrete optimal configurations are **uniform polytopes** (simplices, antiprisms, ...). Elhage et al. [2] shows learned feature geometries *are* such configurations.
- **Discrete Fourier transform / characters of** $\mathbb{Z}/n\mathbb{Z}$: the irreducible representations of the cyclic group are $k \mapsto e^{2\pi i kj/n}$. A network doing modular addition represents inputs on these circles and adds angles (Nanda et al. [7]). The same circular/periodic ("ringing") structure resurfaces geometrically in `when-models-manipulate-manifolds`.
- **Riesz representation**: every continuous linear functional on a Hilbert space is $\langle v, \cdot\rangle$ for a unique v; the map $v \mapsto \langle v, \cdot\rangle$ is the **musical isomorphism** $V \xrightarrow{\sim} V^*$ induced by the in-

ner product — the same identification that turns a **dual map** $W^\vee$ into an **adjoint** W^*. `linear-representation-hypothesis` uses exactly this to identify the embedding-space and unembedding-space representations of a concept under a chosen (non-Euclidean) inner product.

Append new concepts below as later surveys introduce them.

References

[1] N. Elhage, N. Nanda, C. Olsson, et al. *A Mathematical Framework for Transformer Circuits*. Transformer Circuits Thread. 2021. URL: https://transformer-circuits.pub/2021/framework/index.html (visited on 05/29/2026).

[2] N. Elhage, T. Hume, C. Olsson, et al. *Toy Models of Superposition*. Transformer Circuits Thread. 2022. URL: https://transformer-circuits.pub/2022/toy_model/index.html (visited on 05/29/2026).

[3] C. Olsson, N. Elhage, N. Nanda, et al. *In-Context Learning and Induction Heads*. arXiv:2209.11895. Transformer Circuits Thread. 2022. URL: https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html (visited on 05/29/2026).

[4] K. Wang, A. Variengien, A. Conmy, B. Shlegeris, and J. Steinhardt. *Interpretability in the Wild: a Circuit for Indirect Object Identification in GPT-2 small*. arXiv:2211.00593; ICLR 2023. 2022. URL: https://arxiv.org/abs/2211.00593 (visited on 05/29/2026).

[5] N. Elhage et al. *Privileged Bases in the Transformer Residual Stream*. Transformer Circuits Thread. 2023. URL: https://transformer-circuits.pub/2023/privileged-basis/index.html (visited on 05/29/2026).

[6] T. Henighan, S. Carter, T. Hume, et al. *Superposition, Memorization, and Double Descent*. Transformer Circuits Thread. 2023. URL: https://transformer-circuits.pub/2023/toy-double-descent/index.html (visited on 05/29/2026).

[7] N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt. *Progress Measures for Grokking via Mechanistic Interpretability*. arXiv:2301.05217; ICLR 2023. 2023. URL: https://arxiv.org/abs/2301.05217 (visited on 05/29/2026).

[8] E. Ameisen, J. Lindsey, et al. *Circuit Tracing: Revealing Computational Graphs in Language Models*. Transformer Circuits Thread. 2025. URL: https://transformer-circuits.pub/2025/attribution-graphs/methods.html (visited on 05/29/2026).

[9] Anthropic Interpretability Team. *A Toy Model of Interference Weights*. Transformer Circuits Thread. 2025. URL: https://transformer-circuits.pub/2025/interference-weights/index.html (visited on 05/29/2026).

[10] Anthropic Interpretability Team. *A Toy Model of Mechanistic (Un)Faithfulness*. Transformer Circuits Thread. 2025. URL: https://transformer-circuits.pub/2025/faithfulness-toy-model/index.html (visited on 05/29/2026).

[11] A. Jermyn, J. Lindsey, R. Luger, et al. *Progress on Attention*. Transformer Circuits Thread. 2025. URL: https://transformer-circuits.pub/2025/attention-update/index.html (visited on 05/29/2026).

[12] Kamath et al. *Tracing Attention Computation Through Feature Interactions*. Transformer Circuits Thread. 2025. URL: https://transformer-circuits.pub/2025/attention-qk/index.html (visited on 05/29/2026).

[13] J. Lindsey, W. Gurnee, E. Ameisen, et al. *On the Biology of a Large Language Model*. Transformer Circuits Thread. 2025. URL: https://transformer-circuits.pub/2025/attribution-graphs/biology.html (visited on 05/29/2026).